实际编写

目录

- 完整流程
- 将修改实时更新到酒馆
- 让 AI 能看到、操控酒馆页面
- 断点调试错误
- 最终编写流程
- 下一步

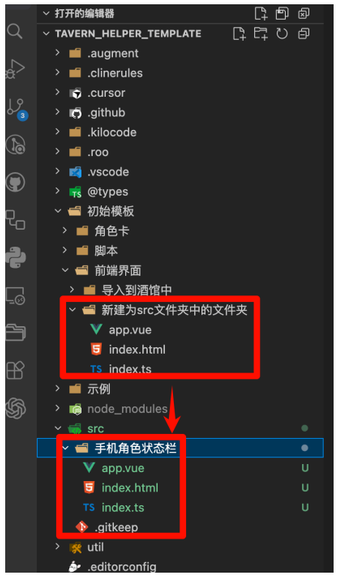
区分了脚本和前端界面, 我们现在可以实际编写它; 如果你不知道酒馆助手提供了哪些功能, 可以查阅酒馆助手文档.

以下内容你都可以先用 示例角色卡 、 示例/脚本示例 和 示例/前端界面示例 先跟着做一遍, 别急着搓自己的东西.
以下内容你都可以先用 示例角色卡 、 示例/脚本示例 和 示例/前端界面示例 先跟着做一遍, 别急着搓自己的东西.
以下内容你都可以先用 示例角色卡 、 示例/脚本示例 和 示例/前端界面示例 先跟着做一遍, 别急着搓自己的东西.

完整流程

在 `初始模板` 文件夹中, 我分别给出了 `脚本` 和 `前端界面` 项目所需的初始文件. 此处以编写 `前端界面` 为例, 而 `脚本` 的编写流程与之类似.

为了创建前端界面项目, 你需要将 `初始模板/前端界面/新建为src文件夹中的文件夹` 复制到 `src` 文件夹中, 将它重命名为你想要的名称:



然后我们发送下面一段提示词给 AI 编程助手, 让它为我们编写代码:

> 阅读现有规则，为我新建一个独立的前端界面`手机角色状态栏`
> 它应该是一个手机风格的角色状态栏，该手机页面应当包含边框、刘海、聊天界面等，并显示有角色当前的状态

> **警告**
>
> 需要注意的是, 我这里只是演示因此直接提出了一个完整要求, 但你在实际编写时应该尽量先和 AI 沟通好, 让 AI 一点点做, 做出来一部分就测试一部分, 而不是指望它能一次性生成完整可用的界面.
>
> 另外, 虽然我给 AI 编程助手添加了全局编写规则, **但它依然可能不阅读**. 你可以把 `.cursor/rules` 中的文档全拖给 AI 来强调它必须阅读.

编写完毕后, 我们在终端执行 `pnpm build` 来打包代码.

如果该操作执行失败, 你可以让 AI 检查错误原因并修复它. 一般直接问 AI 即可, 也可以自己在终端中复制错误信息然后发给 AI:

```
问题 280   输出   调试控制台   终端   端口   GitLens
ERROR in /Users [redacted] sillytavern_data/tavern_helper_template/src/手机角色状态栏/index.ts
./src/手机角色状态栏/index.ts 307:29-318:3
[tsl] ERROR in /Users [redacted] sillytavern_data/tavern_helper_template/src/手机角色状态栏/index.ts(307,30)
      TS2365: Operator '>' cannot be applied to types 'boolean' and '{ return: Promise<unknown>; }'.

ERROR in /Users [redacted] sillytavern_data/tavern_helper_template/src/手机角色状态栏/index.ts
./src/手机角色状态栏/index.ts 307:37-52
[tsl] ERROR in /Users [redacted] sillytavern_data/tavern_helper_template/src/手机角色状态栏/index.ts(307,38)
      TS2693: 'CharacterStatus' only refers to a type, but is being used as a value here.

276 errors have detailed information that is not shown.
Use 'stats.errorDetails: true' resp. '--stats-error-details' to show it.

webpack 5.101.3 compiled with 277 errors in 2191 ms
 ELIFECYCLE  Command failed with exit code 1.
```

如果该操作执行成功, 则会在 `dist` 文件夹中找到 `pnpm build` 对新项目的打包结果:

你只需要考虑打包结果中的那个 .html 文件, 不必考虑另外的 .map 文件.

.html 文件包含了项目中所有文件的代码, 因此你可以**将它的内容用代码块包裹 (上下加一行 ```)**, 然后复制到示例角色卡的 `界面-发布给玩家` 正则中的 `替换为` 部分.

> **提示**
>
> 如果是脚本 (流式楼层界面也是脚本), 则应该使用打包结果中的那个 .js 文件, 将它的内容复制到酒馆助手脚本库脚本中的 `脚本内容` 部分.

这样一来,

- 如果酒馆楼层中存在能被这个酒馆正则匹配的文本, 酒馆正则就会在楼层中替换出代码块; (示例角色卡中的酒馆正则是匹配任意文本, 所以必然能替换出来)
- 而酒馆助手会将代码块渲染成前端界面:

这就是我们项目的整个流程:

1. 在 `src` 文件夹中新建项目
2. 让 AI 在这个项目中编写代码
3. 用 `pnpm build` 将项目打包成一个文件
4. 将 `dist` 文件夹中的打包结果用代码块包裹后复制到酒馆正则中 (如果是脚本 (流式楼层界面也是脚本) 则复制到酒馆助手脚本库脚本中的 `脚本内容` 部分)

但 AI 会听不懂我们的要求或犯错, 一次编写很可能无法写出我们想要的脚本或前端界面. 这又该如何是好呢? 难道我们每次让 AI 修改了代码, 都要重新 `pnpm build` 打包代码, 再手动复制粘贴到酒馆中来验证?

不, 我们更希望:

- 对脚本和前端界面的修改能实时更新到酒馆中
- AI 能自己查看酒馆网页来理解你说的要求或问题
- (有代码基础的人) 能够自行查看代码执行情况来定位错误

将修改实时更新到酒馆

假设我们已经从 `初始模板` 中创建了前端界面项目 `前端界面示例`, 我们希望将它的最新内容实时更新到酒馆中.

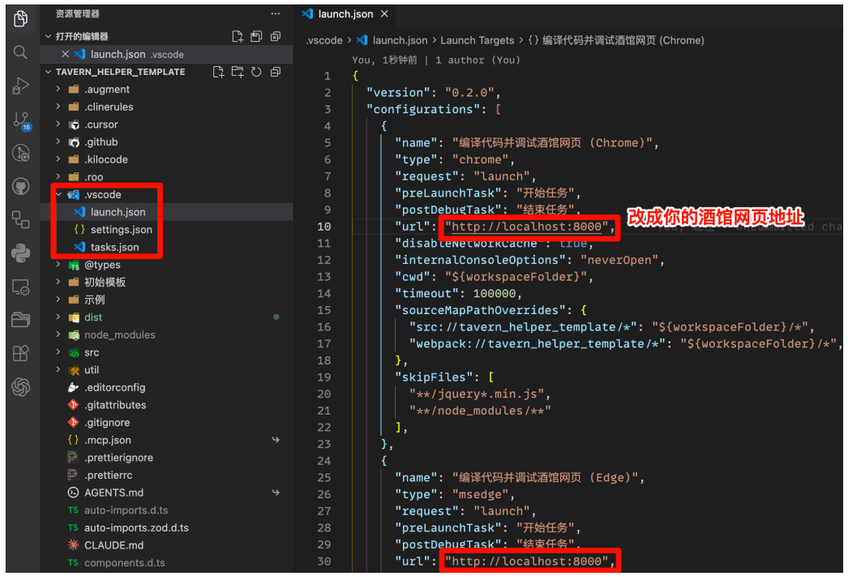
回顾一下如果没有实时修改, 我们是如何修改项目并更新到酒馆中的:

- 让 AI 修改代码
- 重新 `pnpm build` 打包代码
- 将 `dist` 文件夹中的打包结果用代码块包裹后复制到酒馆正则中 (如果是脚本 (流式楼层界面也是脚本) 则复制到酒馆助手脚本库脚本中的 **脚本内容** 部分)

因此, 我们需要实现 "重新 `pnpm build` 打包代码" 和 "将打包结果用代码块包裹后复制到酒馆正则中" 的自动化.

项目被修改后自动打包

要做到项目被修改后自动打包, 我们将 `.vscode/launch.json` 中的 `url` 为自己的酒馆地址 (例如 `http://localhost:6622`).



然后, 我们不再运行 `pnpm build`, 而是运行 `pnpm watch`. watch 意为监听, 也就是说, `pnpm watch` 会监听项目中的文件变化, 并在变化时自动重新打包代码.

将打包结果实时更新到酒馆

为了让酒馆页面实时加载最新的打包结果, 我们不能是再自己手动复制打包结果到酒馆正则中, 而要让酒馆页面自动加载它. 这需要:

- 让酒馆知道打包结果发生了变化并在变化时重新加载脚本和前端界面: 我们需要将酒馆与打包监听器 `pnpm watch` 连接
- 让酒馆在重新加载时能获取到最新打包结果: 我们可以将打包结果文件映射为网络链接, 在酒馆里填写这个网络链接而不是自己每次复制粘贴, 则酒馆重新加载脚本和前端界面时会重新访问这个链接, 从而得到最新的打包结果

将酒馆与打包监听器连接

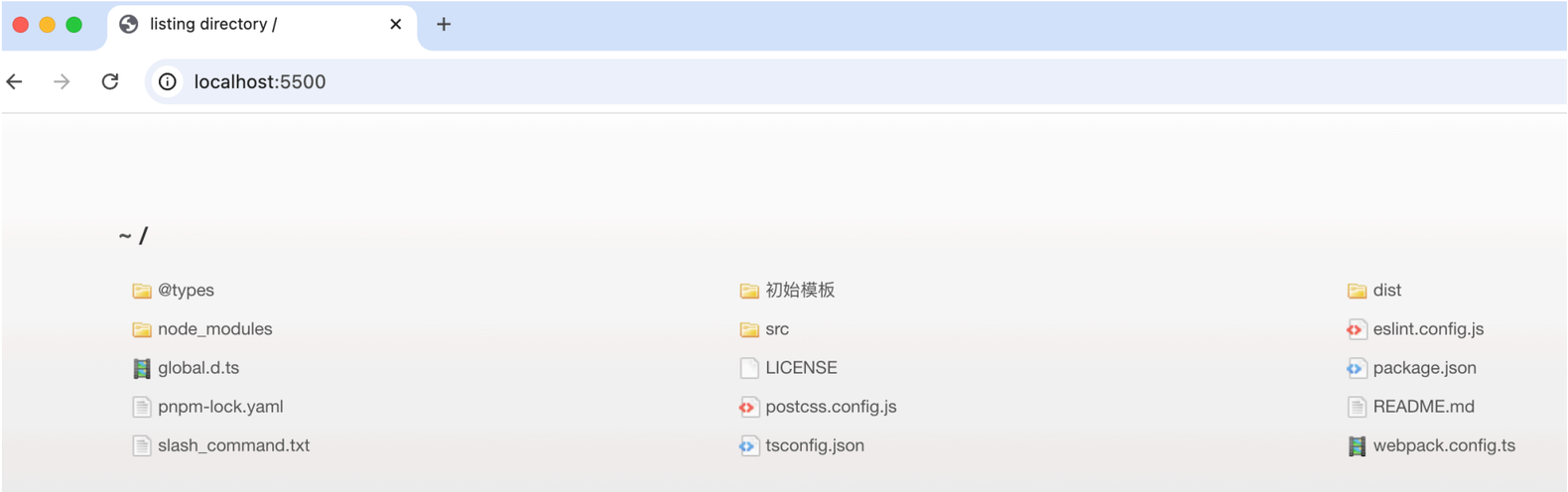
酒馆助手已经为我们实现了酒馆与打包监听器 `pnpm watch` 的连接, 我们从 **酒馆助手 › 开发 › 实时监听** 中启用 **允许监听** 即可.

如果我们已经运行 `pnpm watch`, 则实时监听右边的 WiFi 图标会变成绿色, 这意味着酒馆已经能知道打包结果是否发生变化.

将最新打包结果以链接形式导入

我们点击 Cursor 右下角的 **Go Live** 来将文件夹中所有文件映射成一个网络链接:
(如果没有这个按钮, 请重新导入配置文件; 实在不行可以自己去 **侧边栏 › 扩展** 搜索 `Live Server` 并安装.)

在点击后, **Go Live** 将会变成 **port: xxxx**. 以 `port: 5500` 为例, 这意味着你将能在 `http://localhost:5500` 访问文件夹下的文件:



而打包结果 `dist/前端界面示例/index.html` 将能以 `http://localhost:5500/dist/前端界面示例/index.html` 访问:

有了这个链接, 我们接下来更换酒馆正则 **替换为** 部分的 html 代码块, 让它从 `http://localhost:5500/dist/前端界面示例/index.html` 加载 html:

```
```
<body>
  <script>
    $('body').load('http://localhost:5500/dist/前端界面示例/index.html')
  </script>
</body>
```
```

这正是 `初始模板/前端界面/导入到酒馆` 中的实时修改正则所做的事情, 而 `初始模板/脚本/导入到酒馆` 中的实时修改脚本填法有所不同, 但你不必记忆. 你可以直接从文件夹找到并导入它们, 然后将其中的链接替换为你的打包结果链接.

让 AI 能看到、操控酒馆页面

经过 "将修改实时更新到酒馆", AI 对我们代码的更改就会实时在酒馆上显示了.

但这还不够! 接下来, 让我们配置 MCP, 从而让 AI 能自己查看前端界面、脚本在酒馆网页上是否正常, 能操控酒馆网页, 甚至能够像真人一样玩酒馆 (?!).

(推荐) 选项1: Chrome DevTools MCP

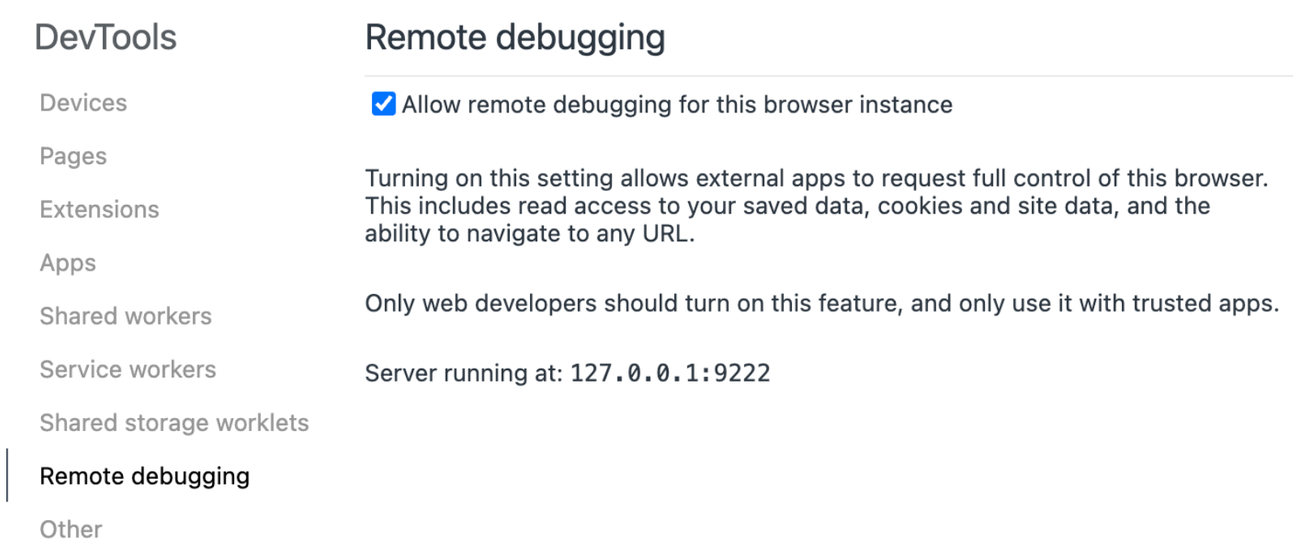
安装 Chrome 浏览器

顾名思义, 我们需要安装 Chrome, 而且最好是最新版.

> **提示**
>
> 网上也有让 Edge 浏览器接上这个功能的方法, 如果需要可以在**学完教程后**去试.

启用远程调试选项

安装好 Chrome 后, 我们在 Chrome 浏览器中输入 `chrome://inspect/#remote-debugging` 并 `回车` 跳转, 然后勾选 **Allow remote debugging for this browser instance** 即可.



如果没有这个选项, 请更新 Chrome 到 144 版本及以上.

在勾选这个选项后, **请通过任务管理器完全关闭浏览器, 或直接重启电脑**, 保证浏览器确实启用了远程调试选项.

添加 MCP

然后, 我们在 AI 编程助手的 MCP 配置文件里添加 Chrome DevTools MCP 即可.....
模板文件夹已经为很多编程助手配置好了它, 但部分编程助手如 Codex 需要自己配置一下. 具体请参考环境准备最后所列出的编程助手配置情况.

配置好后, AI 编程助手就能在编写时自行查看操控酒馆网页了.

你的 AI 编程助手现在可以: (AI 不一定主动使用, 你可以在提示词中告诉它 `用 chrome-devtools-mcp 来访问酒馆网页`)

- 读取酒馆网页显示情况、控制台 (Console) 日志情况、网络情况
- 读取你通过 **开发者工具 › 左上角的选择元素** 选择到的网页元素
- 让它像素级复刻网页样式
- 自己点击网页、填写表单.....甚至如果你说明清楚, 它可以像你一样玩酒馆
- .....

选项2: Browser MCP

或者你可以使用 Browser MCP, 但它比 Chrome DevTools MCP 在使用体验上差了很多.

添加 MCP

首先, 我们来为 AI 编程助手添加 MCP. 我们将 `.cursor/mcp.json` 清空, 粘贴以下内容:

```
{
  "mcpServers": {
    "browsermcp": {
      "command": "pnpx",
      "args": [
        "@browsermcp/mcp"
      ]
    }
  }
}
```

这会为很多编程助手配置好它, 但部分编程助手如 Codex 需要自己配置一下. 具体请参考环境准备最后所列出的编程助手配置情况.

安装浏览器扩展

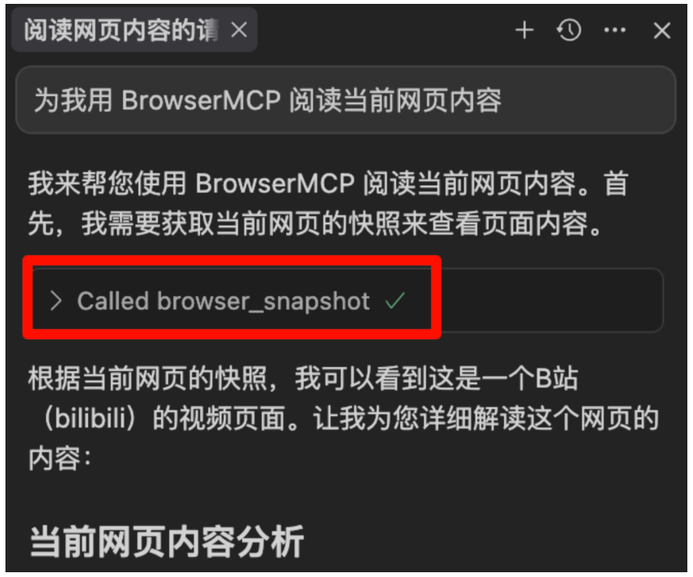
然后, 我们去安装它所需要的浏览器扩展.

我们打开浏览器, 从 chrome 应用商店或你浏览器的应用商店安装最新的浏览器扩展.

如此, 我们回到酒馆网页, 点击扩展图标然后点击 **连接** 即可.

你的 AI 编程助手现在可以: (AI 不一定主动使用, 你可以在提示词中告诉它 `用 browsermcp 来访问酒馆网页`)

- 读取酒馆网页显示情况、控制台 (Console) 日志情况、网络情况
- 读取你通过 **开发者工具 › 左上角的选择元素** 选择到的网页元素



AI 使用 MCP 时会有很明显的显示

断点调试错误

可能太专业因而不详细解释: 这个项目模板也天然支持断点调试. 你可以在代码中设置断点, 然后 `F5` 运行调试任务. 当浏览器运行到断点处时, 会自动暂停并显示运行情况.

最终编写流程

我们现在有了实时修改和 AI 直接看酒馆页面的功能, 则编写流程应该如下:

创建项目

在 `src` 文件夹中用 `初始模板` 新建项目.

在 **酒馆正则** 中导入 `初始模板` 中的实时修改正则 (编写脚本则在 **酒馆助手 › 脚本** 中导入实时修改脚本), 并将其中的链接替换为打包结果链接.

进行编写

- 将修改结果实时更新到酒馆
  1. 点击 Cursor 右下角的 **Go Live** 将文件夹映射为网络链接
  2. 在终端输入 `pnpm watch` 并 `回车` 来运行监听打包任务
- 让 AI 能看到、操控酒馆网页
  - 如果用的是 Chrome DevTools MCP, 什么都不用做直接用
  - 如果用的是 Browser MCP, 需要自己去它的浏览器插件启用连接; 但它只能让 AI 看酒馆网页不能操控
- 在输入框里加一些强调辅助它别发蠢:
  - **强制它阅读规则:** 虽然规则是按 AI 编程助手文档配置的, 但 AI 还是可能偷懒不读, 最好在提示词中强调 `阅读现有规则`
  - **强制它看酒馆:** 同理, 为了防止 AI 偷懒, 你最好强调 `通过 MCP 访问酒馆网页`
  - **告诉它怎么检查结果:** 如果你的界面内容很多, 你可以针对你的需求说明它该怎么检查结果, 比如 `随便填写界面里的表单一直到显示提交按钮, 点击提交按钮后将会出现新的楼层`
- 确认你是要制作什么: 前端界面? 脚本? 流式楼层界面? 一整张 MVU 变量卡?
- 开始提需求让 AI 做, 结果有问题就让它重做或者修改

发布打包结果

在确认项目完成后, 停止 `pnpm watch` 任务 (`Ctrl` + `C` 或直接删除终端窗口), 用 `pnpm build` 将项目打包成一个文件, 将打包结果用代码块包裹后复制到酒馆正则中 (如果是脚本 (流式楼层界面也是脚本) 则复制到酒馆助手脚本库脚本中的 **脚本内容** 部分).

注意你应该发布 `pnpm build` 的打包结果, 而不是 `pnpm watch` 的打包结果. `pnpm watch` 是专门用于实时编写的, 它打包出来的文件大小是 `pnpm build` 的数十倍.

下一步

现在, 你已经可以拷打 AI 编写前端界面或脚本了.

但如果你想知道:

- 酒馆助手让你在前端界面或脚本里能做到什么

- 前端界面正则的推荐写法
- 怎么制作支持流式传输的前端界面
- 怎么制作同层前端界面
- ……

那么请阅读进阶技巧相关部分.